Mistake 1: Forgetting Semicolon (;) at the End of Statements

WRONG:

```
#include <stdio.h>
int main() {
    int x = 5   // ERROR: Missing semicolon
    int y = 10  // ERROR: Missing semicolon
    printf("Sum = %d\n", x + y) // ERROR: Missing semicolon
    return 0;
}
```

CORRECT:

```
#include <stdio.h>
int main() {
    int x = 5;   // Semicolon added
    int y = 10;  // Semicolon added
    printf("Sum = %d\n", x + y); // Semicolon added
    return 0;
}
```

Explanation:

Every statement in C must end with a semicolon (;).

Without it, the compiler cannot tell where one statement ends and the next begins.

This causes a compilation error and the program will not run.

Mistake 2: Using = Instead of == in Comparisons

WRONG:

```
#include <stdio.h>
int main() {
    int age = 18;
    if (age = 20) {          // WRONG: = assigns, does not compare
        printf("Age is 20\n"); // This always runs!
    }
    return 0;
}
```

CORRECT:

```
#include <stdio.h>
int main() {
    int age = 18;
    if (age == 20) {         // CORRECT: == compares values
        printf("Age is 20\n");
    } else {
        printf("Age is NOT 20\n"); // This runs correctly
    }
    return 0;
}
```

Explanation:

= (single equal) -> ASSIGNMENT: assigns a value to a variable.

== (double equal) -> COMPARISON: checks if two values are equal.

Using = inside if() assigns the value and the condition becomes TRUE (non-zero), causing the block to always execute regardless of the original value.

Mistake 3: Forgetting & in scanf()

WRONG:

```
#include <stdio.h>
int main() {
    int number;
    printf("Enter a number: ");
    scanf("%d", number);  // ERROR: Missing &
    printf("You entered: %d\n", number);
    return 0;
}
```

CORRECT:

```
#include <stdio.h>
int main() {
    int number;
    printf("Enter a number: ");
    scanf("%d", &number); // CORRECT: & added before number
    printf("You entered: %d\n", number);
    return 0;
}
```

Explanation:

scanf() needs to STORE the value entered by the user into the variable.
To do that, it needs the MEMORY ADDRESS of the variable, not the variable itself.

& (address-of operator) gives scanf() the memory address of the variable.
Without &, scanf() receives a garbage value and the program may CRASH.

Mistake 4: Array Index Out of Bounds

WRONG:

```
#include <stdio.h>
int main() {
    int nums[5] = {10, 20, 30, 40, 50};
    // Valid indexes: 0, 1, 2, 3, 4
    printf("%d\n", nums[5];  // ERROR: Out of bounds
    printf("%d\n", nums[10]); // ERROR: Way out of bounds
    return 0;
}
```

CORRECT:

```
#include <stdio.h>
int main() {
    int nums[5] = {10, 20, 30, 40, 50};
    printf("%d\n", nums[0]); // Output: 10
    printf("%d\n", nums[4]); // Output: 50
    for (int i = 0; i < 5; i++) {
        printf("nums[%d] = %d\n", i, nums[i]);
    }
    return 0;
}
```

Explanation:

An array of size 5 has valid indexes 0 to 4 only.

Accessing index 5 or beyond goes into memory that does NOT belong to the array.

This causes:

-> Garbage / unpredictable output

-> Program crashes (Segmentation Fault)

-> Dangerous memory corruption

Rule: For array[n], valid indexes = 0 to n-1

Mistake 5: Not Returning a Value from a Non-void Function

WRONG:

```
#include <stdio.h>

// ERROR: Function says it returns int but has no return statement
int addNumbers(int a, int b) {
    int sum = a + b;
    // Missing: return sum;
}

int main() {
    int result = addNumbers(4, 6);
    printf("Result = %d\n", result); // Prints garbage!
    return 0;
}
```

CORRECT:

```
#include <stdio.h>

// CORRECT: Function returns the computed value
int addNumbers(int a, int b) {
    int sum = a + b;
    return sum; // Value is returned
}

int main() {
    int result = addNumbers(4, 6);
    printf("Result = %d\n", result); // Output: 10
    return 0;
}
```

Explanation:

If a function is declared to return a type (int, float, char, etc.),
it MUST have a return statement.

Without return:
-> Compiler gives a WARNING
-> Function returns a garbage / random value
-> Program produces wrong results

EXCEPTION: void functions do NOT need a return statement.
void printHello() { printf("Hello!\n"); } <- No return needed